

### บทที่ 3

### ทฤษฎีและหลักการเกม 3 มิติ

ในการพัฒนาโปรแกรมเกม 3 มิติ จำเป็นอย่างยิ่งที่จะต้องมีความเข้าใจถึงทฤษฎีพื้นฐานที่จำเป็นสำหรับนำไปประยุกต์ใช้ โดยเฉพาะทฤษฎีทางด้านคอมพิวเตอร์กราฟิก ซึ่งเป็นพื้นฐานสำหรับการพัฒนาโปรแกรมเกม 3 มิติ

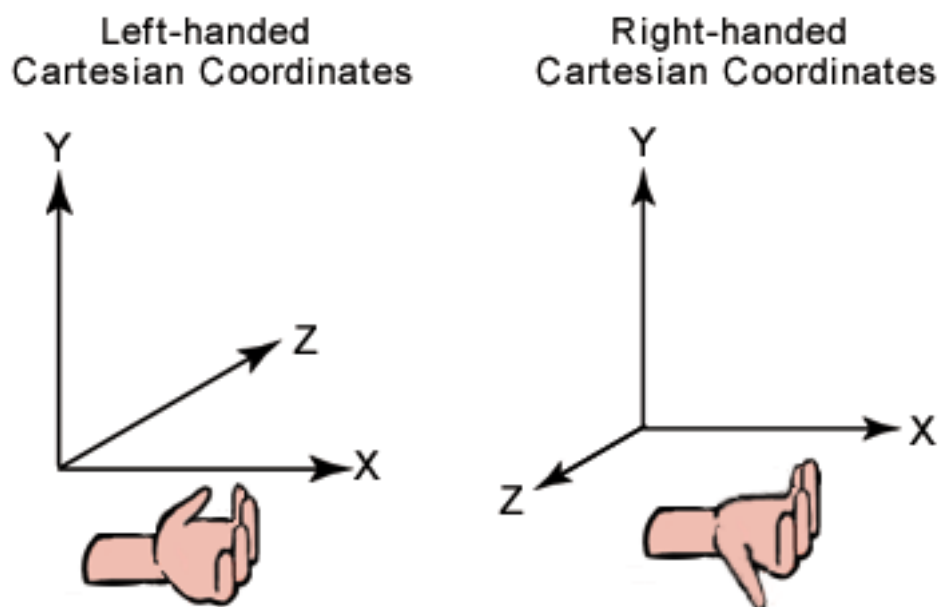
#### 3. ระบบพิกัด 3 มิติ

##### 3.1.1 ระบบพิกัดคาร์ทีเซียน (Cartesian Coordinate System)

เป็นระบบพิกัดที่ทำความเข้าใจได้ง่ายที่สุด เพราะเป็นระบบพิกัดที่คนส่วนใหญ่คุ้นเคย และมีใช้อย่างแพร่หลายที่สุด อีกทั้งยังเหมาะสมในการใช้สำหรับสร้างโปรแกรม ระบบพิกัดคาร์ทีเซียนประกอบด้วยสามแกนที่ตั้งฉากซึ่งกันและกันสำหรับกำหนดพิกัด โดยมักจะตั้งชื่อแกนนดังกล่าวให้เป็น X, Y และ Z ระบบพิกัดนี้โดยทั่วไปมักมีการกำหนดแนวแกนได้ 2 แบบคือ

1. ระบบพิกัดคาร์ทีเซียนแบบมือซ้าย (Left-handed Cartesian Coordinate System)
2. ระบบพิกัดคาร์ทีเซียนแบบมือขวา (Right-handed Cartesian Coordinate System)

โดยแสดงได้ดังภาพดังต่อไปนี้

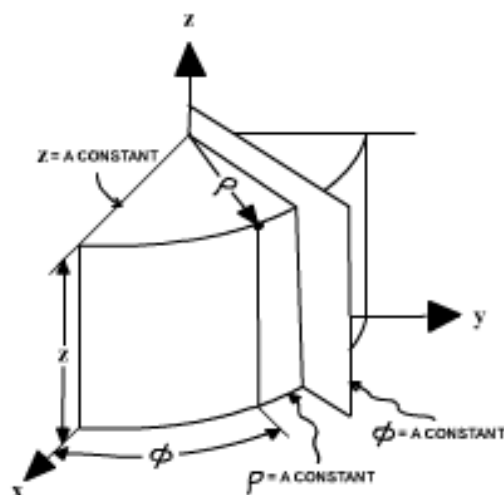


รูปที่ 3-1 แสดงระบบพิกัดคาร์ทีเซียนแบบมือซ้ายและแบบมือขวา

โดยทั่วไป DirectX Graphics API นั้นระบบพิกัดที่ใช้จะเป็นระบบมือซ้าย ในการแสดงผลภาพ 3 มิติ นั้นเราจะต้องส่งเวอร์เทกซ์อาร์เรย์ที่เราต้องการแสดงผลเข้าไปใน Geometry Pipeline สุดท้ายเราก็จะได้ภาพ 2 มิติออกมาแสดงบนจอภาพ เหตุที่เราต้องแปลงข้อมูลที่เป็น 3 มิติให้เป็น 2 มิติ นั้นเพราะหน้าจอของเราไม่สามารถแสดงผลภาพ 3 มิติตรงๆ ได้จึงต้องทำการส่งข้อมูลเข้า Geometry Pipeline เพื่อให้ได้ผลลัพธ์เป็นภาพ 2 มิติที่สอดคล้องกับข้อมูลนั้นบนมุมมองที่ต้องการ

### 3.1.2 ระบบพิกัดทรงกระบอก (Cylindrical Coordinate System)

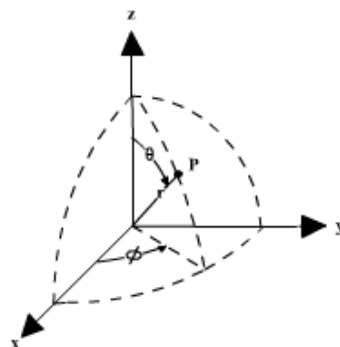
ในระบบพิกัดนี้ เราสามารถระบุพิกัดของจุดแต่ละจุดได้โดยใช้ระยะห่างระหว่างจุดนั้นๆ กับจุดกำเนิด (Origin) ซึ่งแทนด้วยสัญลักษณ์  $\rho$  มุมระหว่างแกนอ้างอิงในแนวระดับ (มักสัมพันธ์กับแกน X และระนาบ XY ในระบบพิกัดคาร์ทีเซียน) กับเส้นตรงที่ลากจากจุดกำเนิดไปยังตำแหน่งของจุดนั้นๆ ในระนาบของจุดกำเนิดกับแกนอ้างอิงในแนวระดับ (ซึ่งสัมพันธ์กับระนาบ XY ในระบบพิกัดคาร์ทีเซียน) ซึ่งแทนด้วยสัญลักษณ์  $\phi$  และระดับความสูงของจุดนั้นๆ ซึ่งมักแทนด้วย Z ดังแสดงในรูป



รูปที่ 3-2 แสดงระบบพิกัดทรงกระบอก

### 3.1.3 ระบบพิกัดทรงกลม (Spherical Coordinate System)

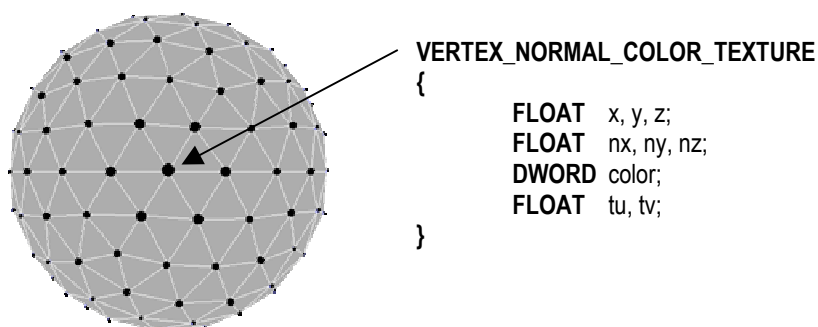
ในระบบพิกัดนี้ เราสามารถระบุพิกัดของจุดแต่ละจุดได้โดยใช้ระยะห่างระหว่างจุดนั้นๆ กับจุดกำเนิด ซึ่งแทนด้วยสัญลักษณ์  $r$  มุมระหว่างแกนอ้างอิงในแนวระดับ (มักสัมพันธ์กับแกน X และระนาบ XY ในระบบพิกัดคาร์ทีเซียน) กับเส้นตรงที่ลากจากจุดกำเนิดไปยังตำแหน่งของจุดนั้นๆ ในระนาบของจุดกำเนิดกับแนวระดับ (ซึ่งสัมพันธ์กับระนาบ XY ในระบบพิกัดคาร์ทีเซียน) ซึ่งแทนด้วยสัญลักษณ์  $\phi$  และมุมระหว่างแกนตั้งฉากแนวระดับ (แกน Z) กับเส้นตรงที่ลากจากจุดกำเนิดไปยังตำแหน่งของจุดนั้นๆ ดังแสดงในรูป



รูปที่ 3-3 แสดงระบบพิกัดทรงกลม

### 3.2 เวอร์เท็กซ์ (Vertex)

เวอร์เท็กซ์เป็นจุดพิกัดในระบบ 3 มิติ ซึ่งใช้ในการอ้างอิงตำแหน่ง ซึ่งในการประมวลผลทางด้านการกราฟิกนั้นเรายังจะกำหนดคุณสมบัติเพิ่มเติมให้กับเวอร์เท็กซ์ เช่น Color, Normal, Texture Coordinate เป็นต้น ซึ่งคุณสมบัติเหล่านี้เมื่อประกอบกับตำแหน่งของเวอร์เท็กซ์แล้วจะนำไปใช้ในการเก็บข้อมูลที่แทนแบบจำลอง 3 มิติได้ โดยแสดงได้ดังรูปตัวอย่าง

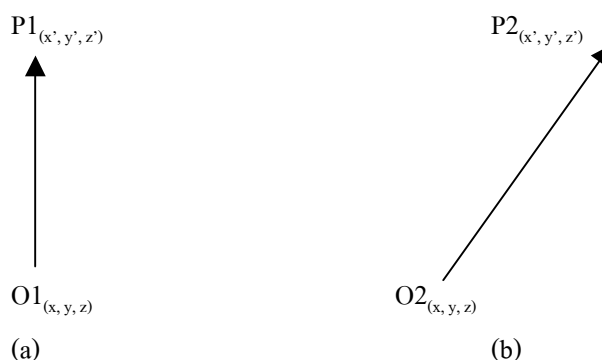


รูปที่ 3-4 แสดงตัวอย่างการนำเวอร์เท็กซ์มาใช้ในการเก็บข้อมูลที่แทนแบบจำลอง 3 มิติ

### 3.3 เวกเตอร์ (Vector)

เวกเตอร์เป็นปริมาณที่มีทั้งขนาดและทิศทาง เป็นองค์ประกอบทางคณิตศาสตร์ที่มีความสำคัญมากด้านการประยุกต์ใช้ในงานการประมวลผลกราฟิก ซึ่งจะต่างกับปริมาณสเกลาร์ตรงที่เวกเตอร์นั้นจะมีทิศทาง หรือกล่าวได้อีกอย่างหนึ่งว่าเวกเตอร์มีองค์ประกอบมาจากปริมาณสเกลาร์  $n$  ตัว เพื่อแทนขนาดและทิศทางในระบบ  $n$  มิติ

เรามักจะแทนเวกเตอร์โดยใช้สัญลักษณ์  $\overrightarrow{OP}$  โดยหมายถึงเวกเตอร์นี้มีทิศทางจากจุด O ไปยังจุด P และมีขนาดเท่ากับระยะห่างระหว่างจุด O และจุด P



รูปที่ 3-5 แสดงตัวอย่างเวกเตอร์

หากเราทำการบวกเวกเตอร์ทั้งสองจะได้ค่าดังนี้

$$R = V1 + V2$$

$$R = (V1_x + V2_x, V1_y + V2_y, V1_z + V2_z)$$

เมื่อทำการคูณเวกเตอร์ด้วยปริมาณสเกลาร์ ซึ่งทำได้โดยการคูณปริมาณสเกลาร์กับทุกคอมโพเนนต์ของเวกเตอร์นั้น ๆ จะได้

$$V * s = (V_x * s, V_y * s, V_z * s)$$

สำหรับขนาด (Magnitude/length) ของเวกเตอร์ใดๆ นั้น สามารถเขียนแทนด้วย  $|V|$  ซึ่งสามารถหาได้โดยใช้กฎของพีทาโกรัส ได้สมการดังนี้

$$|V| = \sqrt{V_x^2 + V_y^2 + V_z^2}$$

เวกเตอร์ที่ขนานกัน (Parallel Vector) หมายถึง เวกเตอร์คู่ใด ๆ ที่มีทิศทางเดียวกันหรือทิศทางตรงข้ามกัน

เวกเตอร์ร่วมระนาบ (Coplanar Vector) หมายถึง เวกเตอร์ตั้งแต่ 3 ตัวขึ้นไปที่อยู่ในระนาบเดียวกัน

### 3.4 เมทริกซ์และทรานส์ฟอร์มเมชัน 3 มิติ

#### 3.4.1 เมทริกซ์ (Matrix)

เมทริกซ์มิติ  $m \times n$  ประกอบด้วยจำนวนจริงที่เขียนเรียงเป็นแถว (Row)  $m$  แถว และเขียนในแนวดิ่ง  $n$  หลัก (Column) โดยปิดล้อมจำนวนจริงเหล่านี้ด้วยเครื่องหมาย [ ] หรือ ( ) จำนวนแต่ละจำนวนในเมทริกซ์ เรียกว่า สมาชิกของเมทริกซ์

$$\begin{bmatrix} 2 & 1 & 0 \\ 5 & 3 & 2 \end{bmatrix} \quad \text{เป็นเมทริกซ์มิติ } 2 \times 3 \quad \begin{bmatrix} 1 \\ 0 \\ 2 \\ 4 \end{bmatrix} \quad \text{เป็นเมทริกซ์มิติ } 4 \times 1$$

ถ้าเมทริกซ์ที่มีจำนวนแถวและจำนวนหลักเท่ากันเท่ากับ  $n$  จะเรียกเมทริกซ์นั้นว่า เมทริกซ์จัตุรัสมิติ  $n$  และเรียก  $a_{11}, a_{12}, \dots, a_{nn}$  ว่าเป็นสมาชิกในแนวทแยงของ  $A$  เมื่อ  $A$  เป็นเมทริกซ์จัตุรัสมิติ  $n$

$a_{11}, a_{22}, a_{33}$  เป็นสมาชิกในแนวทแยงของ  $A$  หากเมทริกซ์จัตุรัส  $A = [a_{ij}]$  ซึ่งมีสมาชิกทุกตัวนอกจากแนวทแยงเป็นศูนย์ทั้งหมด (นั่นคือ  $a_{ij} = 0$  ถ้า  $i \neq j$ ) เรียก  $A$  ว่าเป็น เมทริกซ์ทแยง (Diagonal Matrix)

ให้  $A = [a_{ij}]_{m \times n}$  และ  $B = [b_{ij}]_{m \times n}$  เป็นเมทริกซ์ที่มีมิติเท่ากัน เรียก  $A$  เท่ากับ  $B$  ก็ต่อเมื่อ  $a_{ij} = b_{ij}$  สำหรับ  $1 \leq i \leq m, 1 \leq j \leq n$  และเขียนแทนด้วย  $A = B$

ถ้า  $A = [a_{ij}]_{m \times n}$  และ  $B = [b_{ij}]_{m \times n}$  แล้วผลบวกของ  $A$  และ  $B$  เขียนแทนด้วย  $A + B$  หมายถึง  $C = [c_{ij}]_{m \times n}$  ซึ่ง

$$c_{ij} = a_{ij} + b_{ij} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

ให้  $A = [a_{ij}]_{m \times n}$  เป็นเมทริกซ์ และ  $k$  เป็นจำนวนจริงใดๆ แล้ว ผลคูณสเกลาร์  $kA$  จะเป็นเมทริกซ์  $[ka_{ij}]_{m \times n}$  เช่น

$$\text{ให้ } A = \begin{bmatrix} 2 & 1 & 0 \\ 5 & 3 & 2 \end{bmatrix} \text{ และ } k = 3$$

$$\text{ดังนั้น } kA = \begin{bmatrix} 3(2) & 3(1) & 3(0) \\ 3(5) & 3(3) & 3(2) \end{bmatrix} = \begin{bmatrix} 6 & 3 & 0 \\ 15 & 9 & 6 \end{bmatrix}$$

ให้  $A = [a_{ij}]_{m \times p}$  และ  $B = [b_{ij}]_{p \times n}$  แล้ว ผลคูณของ A และ B เขียนแทนด้วย AB หมายถึง

$$c = [a_{ij}]_{m \times n} \text{ โดย } c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{ip}b_{pj}$$

$$c = \sum_{k=1}^p a_{ik} b_{kj} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

เช่น  $AB = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix}$

$$= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} & a_{11}b_{13} + a_{12}b_{23} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} & a_{21}b_{13} + a_{22}b_{23} \end{bmatrix}$$

ผลคูณ AB ไม่มีหรือไม่นิยาม (Undefined) ถ้า A เป็นเมทริกซ์ขนาด  $m \times p$  และ B เป็นเมทริกซ์ขนาด  $q \times n$  เมื่อ  $p \neq q$

การคูณเมทริกซ์นั้นไม่มีคุณสมบัติการสลับที่ซึ่งหมายถึง  $A \times B \neq B \times A$  เมื่อ A และ B เป็นเมทริกซ์จัตุรัส

เมทริกซ์เอกลักษณ์ (Identity Matrix) คือ เมทริกซ์จัตุรัสที่มีสมาชิกในแนวทแยงเป็น 1 ทั้งหมด ซึ่งเขียนแทนด้วย I หรือ  $I_n$  แทนเมทริกซ์เอกลักษณ์มิติ n เช่น

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

ถ้า A เป็นเมทริกซ์มิติ  $m \times n$  แล้วจะได้ว่า  $AI_n = A$  และ  $I_m A = A$

B เป็นอินเวอร์สการคูณของเมทริกซ์ A (B เป็นอินเวอร์สของ A) เมื่อ B เป็นเมทริกซ์ซึ่ง  $AB = BA = I$  โดยที่ A เป็นเมทริกซ์จัตุรัสใดๆ

ให้ A เป็นเมทริกซ์จัตุรัสมิติ n จะกล่าวว่า A มีตัวผกผัน (Invertible) หรือ A เป็นเมทริกซ์ซึ่งมิใช่เอกฐาน (Non-Singular Matrix) ถ้าสามารถหาเมทริกซ์จัตุรัส B ได้ ซึ่งทำให้

$$AB = I_n = BA$$

และเรียกเมทริกซ์ B ว่าเป็นอินเวอร์สการคูณ ของ A เขียนแทนด้วยสัญลักษณ์  $A^{-1}$  นั่นคือ ถ้า A เป็นเมทริกซ์ซึ่งมิใช่เมทริกซ์เอกฐานมิติ n แล้ว จะได้ว่า

$$AA^{-1} = A^{-1}A = I_n$$

และถ้า A ไม่มีอินเวอร์ส แล้วจะเรียก A ว่าเป็นเมทริกซ์เอกฐาน (Singular Matrix)

### 3.4.2 Translation

เป็นการเคลื่อนย้ายตำแหน่งของวัตถุ ซึ่งใช้ Translation Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ T_x & T_y & T_z & 1 \end{bmatrix}$$

Translation Matrix

### 3.4.3 Rotation

เป็นการหมุนวัตถุรอบแกน X, Y หรือ Z ซึ่งใช้ Rotation Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation Around X Axis Matrix

$$\begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation Around Y Axis Matrix

$$\begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation Around Z Axis Matrix

### 3.4.4 Scaling

เป็นการย่อหรือขยายวัตถุ ซึ่งใช้ Scaling Matrix

$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

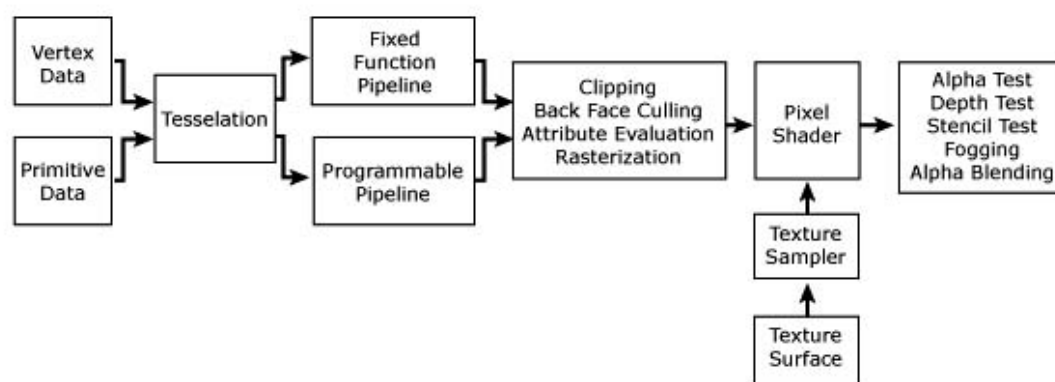
Scaling Matrix

## 3.5 การแสดงผลภาพ 3 มิติด้วย Direct3D

### 3.5.1 สถาปัตยกรรมของ Direct3D

สถาปัตยกรรมของ Direct3D ประกอบด้วยหน้าที่การทำงานต่างๆ ซึ่งจะรวมกันทำงานแบบ Pipeline ดังที่แสดงตามรูป

#### Graphics Pipeline



รูปที่ 3-6 รูปแสดง Pipeline การทำงานของ Direct3D



### 3.5.2 ทรานส์ฟอร์มเมชัน (Transformation)

#### 3.5.2.1 การแปรไปสู่พิกัดเวิลด์ (World Transformation)

ขั้นตอนนี้จะเป็นขั้นตอนในการแปลง Local Coordinate ไปเป็น World Coordinate ซึ่งจะเป็นตำแหน่งจริงๆ ในปริภูมิ 3 มิติ ซึ่งวิธีการนั้นเราจะใช้การคูณตำแหน่งในเวอร์เท็กซ์ด้วย Transformation Matrix ต่าง ๆ เพื่อให้ได้ตำแหน่งที่เราต้องการ

ในการแปลงเวอร์เท็กซ์จาก Local Coordinate ให้เป็น World Coordinate นั้นในบางครั้งถ้าการแปลงของเรามีความซับซ้อนมากเราอาจจะต้องใช้เมทริกซ์หลายตัวในการแปลงคูณต่อกันไป (Matrix Concatenation) เพื่อสร้างเมทริกซ์สุดท้ายสำหรับใช้เป็น Transformation Matrix ก่อนแล้วจึงจะเอามาคูณกับตำแหน่งในเวอร์เท็กซ์จริงเพื่อให้ได้ผลลัพธ์สุดท้ายก็ได้

การคูณ Matrix กับตำแหน่งของ Vertex จะเป็นดังต่อไปนี้

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} M_{11} & M_{12} & M_{13} & M_{14} \\ M_{21} & M_{22} & M_{23} & M_{24} \\ M_{31} & M_{32} & M_{33} & M_{34} \\ M_{41} & M_{42} & M_{43} & M_{44} \end{bmatrix}$$

$$\begin{aligned} x' &= (x \times M_{11}) + (y \times M_{21}) + (z \times M_{31}) + (1 \times M_{41}) \\ y' &= (x \times M_{12}) + (y \times M_{22}) + (z \times M_{32}) + (1 \times M_{42}) \\ z' &= (x \times M_{13}) + (y \times M_{23}) + (z \times M_{33}) + (1 \times M_{43}) \end{aligned}$$

จากรูป  $x, y, z$  คือ ตำแหน่งของเวอร์เท็กซ์พอยกับเมทริกซ์ดังรูปแล้ว จะได้ผลลัพธ์ เป็น  $x', y', z'$  ซึ่งเป็นผลลัพธ์ของการแปลงพิกัด

#### 3.5.2.2 การแปรไปสู่พิกัดวิว (View Transformation)

ขั้นตอนนี้ใช้สำหรับแปลง World Coordinate ให้สอดคล้องกับการเห็นของผู้สังเกต เนื่องจาก World Coordinate บอกเพียงตำแหน่งจริงๆ ในพิกัด 3 มิติเท่านั้น ยังไม่มีการอ้างอิงจากผู้สังเกตเลย ดังนั้นจึงต้องมีการบอก ว่าผู้สังเกตอยู่ที่ไหน และมองไปทางไหน ขั้นตอนนี้มักใช้ในการปรับมุมมองของกล้อง โดยการนำวิวมเมทริกซ์มาคูณกับ World Coordinate จากการแปรไปสู่พิกัดเวิลด์

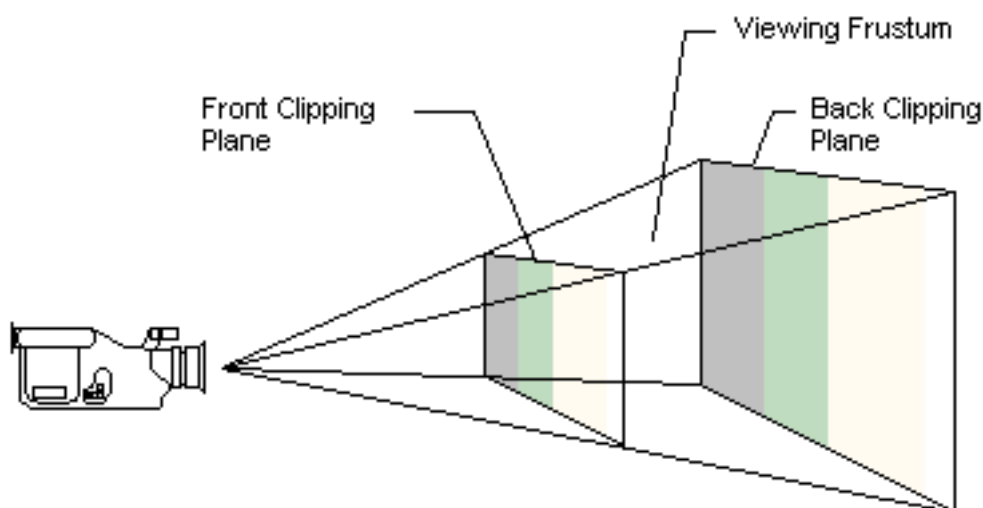
$$\begin{bmatrix} u_x & v_x & n_x & 0 \\ u_y & v_y & n_y & 0 \\ u_z & v_z & n_z & 0 \\ -(u \cdot c) & -(v \cdot c) & -(n \cdot c) & 1 \end{bmatrix}$$

จากรูปวิวเมทริกซ์ประกอบด้วยเวกเตอร์ 3 ตัวบอกทิศทางของผู้สังเกตว่าหันหน้าไปทางใด ซึ่งมีเวกเตอร์  $u$ ,  $v$ ,  $n$  บอกถึง Up, Right, และ View Direction ตามลำดับ และเวกเตอร์  $c$  ซึ่งบอกตำแหน่งของผู้สังเกตในพิกัด World

วิวเมทริกซ์นั้นให้ข้อมูลที่จำเป็นในการปรับมุมมองของกล้องในเกมเอนจินได้ ซึ่งใช้ในการบอกตำแหน่งและทิศทางของกล้อง

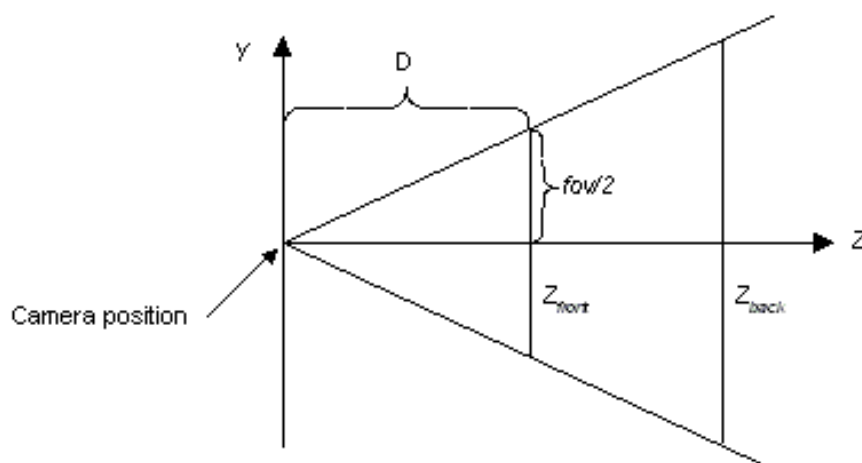
### 3.5.2.3 การแปลงไปสู่พิกัดโปรเจกต์ชัน (Projection Transformation)

เมื่อเราได้ตำแหน่งซึ่งอ้างอิงกับผู้สังเกตเรียบร้อยแล้ว ขั้นตอนต่อไปก็คือการแปลง View Coordinate มาเป็นพิกัด 2 มิติ ซึ่งสามารถนำมาวาดได้จริงๆ บนจอภาพ ซึ่งโดยทั่วไปแล้วเราต้องการให้ได้ภาพที่สมจริง คือ วัตถุที่อยู่ไกลจากสายตาจะมีขนาดเล็กลง และวัตถุที่อยู่ใกล้สายตาจะมีขนาดใหญ่ขึ้น ซึ่งเรียกว่า Perspective Projection Transformation ซึ่งเป็นการสร้างภาพฉายมาดกกระทบฉาก คล้ายๆ กับการฉายภาพจากโปรเจกเตอร์

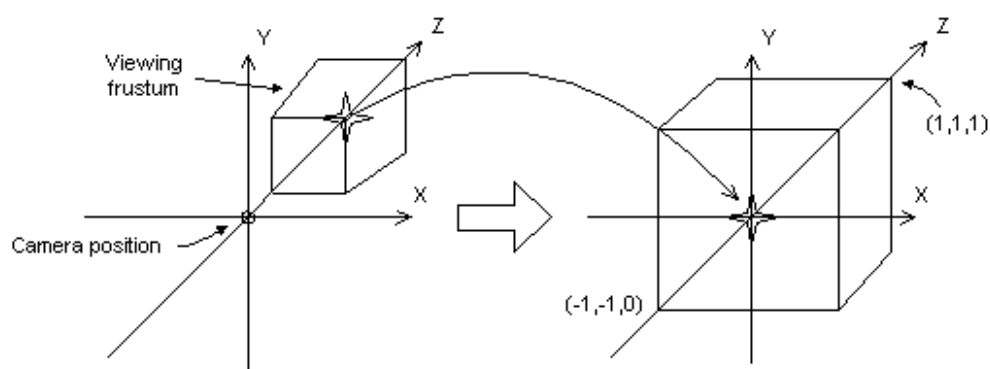


รูปที่ 3-7 Viewing Frustum

วัตถุที่อยู่ใน Viewing Frustum จะถูกนำมาแสดงผล เนื่องจากเป็นส่วนที่จอภาพสามารถเห็นได้ ส่วนวัตถุที่อยู่ภายนอก Viewing Frustum จะถูกตัดออกระหว่างการประมวลผลก่อนที่จะนำไปแสดงผล



รูปที่ 3-8 Viewing Frustum มองจากแนวแกน X



รูปที่ 3-9 การแปลงจาก Viewing Frustum (จากภาพด้านซ้าย) ไปเป็น Cuboid shape (จากภาพด้านขวา) ซึ่งทำให้วัตถุที่อยู่ใกล้มีขนาดเล็กลง และวัตถุที่อยู่ไกลมีขนาดใหญ่ขึ้นตามสัดส่วนของระยะห่างจากสายตาผู้สังเกต

เราสามารถทำ Perspective Projection Transformation ได้โดยใช้ Perspective Projection Matrix ดังนี้

$$\begin{bmatrix} w & 0 & 0 & 0 \\ 0 & h & 0 & 0 \\ 0 & 0 & Q & 1 \\ 0 & 0 & -QZ_n & 0 \end{bmatrix}$$

$$w = \cot\left(\frac{fov_w}{2}\right)$$

$$h = \cot\left(\frac{fov_h}{2}\right)$$

$$Q = \frac{Z_f}{Z_f - Z_n}$$

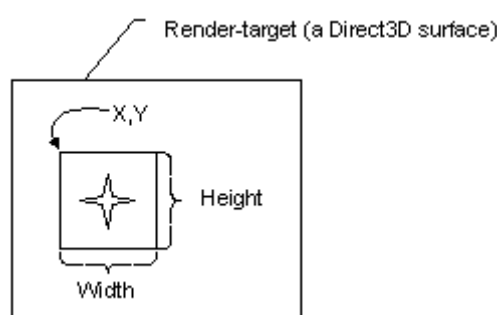
โดยทั่วไปแล้วกราฟิกเอนจินนั้นมักจะมีฟังก์ชันที่ใช้สำหรับการคำนวณหา Perspective Projection Matrix โดยให้เรากำหนดค่า Field of View (fov) ตามแนวความสูงของฉาก (โดยทั่วไปมักเป็น แกน Y) ค่า Aspect Ratio (w/h) ค่า Near Clipping Plane ( $Z_n$ ) และค่า Far Clipping Plane ( $Z_f$ )

นอกจากการทำ Perspective Projection Transformation แล้วยังมีการทำ Projection Transformation แบบอื่นๆ อีก เช่น Isometric Projection Transformation, Oblique Projection Transformation เป็นต้น

### 3.5.3 Clipping and Viewport Scaling

Viewport ในที่นี้หมายถึงพื้นที่สี่เหลี่ยมผืนผ้าที่ใช้ในการฉายภาพที่เกิดจากการแปรไปสู่พิกัดโปรเจกต์ชัน ซึ่งเป็นขั้นตอนสุดท้ายใน Geometry Pipeline และทำให้ได้ภาพฉายตกลงบน Viewport เพื่อทำกระบวนการ Rasterization ต่อไปในการสร้างภาพเสมือนจริงบนหน้าจอ

Viewport อาจหมายถึงหน้าจอทั้งหน้าจอหรือบางส่วนของหน้าจอก็ได้ ทั้งนี้ขึ้นอยู่กับโปรแกรม ซึ่งโดยทั่วไปแล้วเกมมักจะแสดงผลแบบเต็มจอ แต่การใช้ Viewport อาจมีประโยชน์เมื่อเราต้องการการแสดงผล 3 มิติหลายหน้าต่าง



รูปที่ 3-10 Direct3D Viewport

โดยทั่วไปแล้วเราจะทำการขริบ (Clipping) สิ่งที่ไม่เห็นบนหน้าจอออกไป ในกรณีที่ Viewport แสดงถึงบางส่วนของหน้านั้น เราจะขริบส่วนของภาพที่อยู่นอกกรอบ Viewport ออกไป ทั้งนี้รวมถึงการขริบในแนวแกน Z ด้วย (เราจะตัดทอนส่วนของภาพที่อยู่ไกลเกิน Viewport ที่กำหนด หรือส่วนของภาพที่อยู่ด้านหลัง Near Clipping Plane ของ Viewing Frustum ออก)

กระบวนการนี้โดยมากมักจัดการโดยกราฟิกเอนจิน โดยเราเพียงแค่กำหนดค่าขอบเขตของ Viewport และระยะการตัดทอนตามแนวแกน Z ก็เพียงพอแล้ว